

Лабораторна робота №3

Маніпулювання даними SQL / OLTP-запити

Виконав: Гасюк Д.О.

Група: ІО-41

Тема проєкту: База даних Telegram-бота для продажу цифрових товарів

Мета роботи:

Метою лабораторної роботи було практичне виконання SQL-запитів для маніпулювання даними в PostgreSQL. У роботі використовуються OLTP-запити, які імітують типові короткі транзакційні операції: отримання даних, додавання нових записів, оновлення існуючих записів і безпечне видалення записів.

У межах роботи було виконано запити типу:

SELECT - для вибірки даних з таблиць;

INSERT - для додавання нових записів;

UPDATE - для зміни існуючих записів;

DELETE - для видалення записів з використанням умови WHERE.

Початкова перевірка даних:

Перед виконанням OLTP-запитів було перевірено кількість записів у таблицях бази даних:

```
SELECT 'bot_user' AS table_name, COUNT(*) AS rows_count FROM bot_user  
UNION ALL
```

```
SELECT 'product', COUNT(*) FROM product  
UNION ALL
```

```
SELECT 'promo_code', COUNT(*) FROM promo_code  
UNION ALL
```

```
SELECT 'bot_command', COUNT(*) FROM bot_command  
UNION ALL
```

```
SELECT 'referral', COUNT(*) FROM referral  
UNION ALL
```

```
SELECT 'publication', COUNT(*) FROM publication  
UNION ALL
```

```
SELECT 'bot_order', COUNT(*) FROM bot_order  
UNION ALL
```

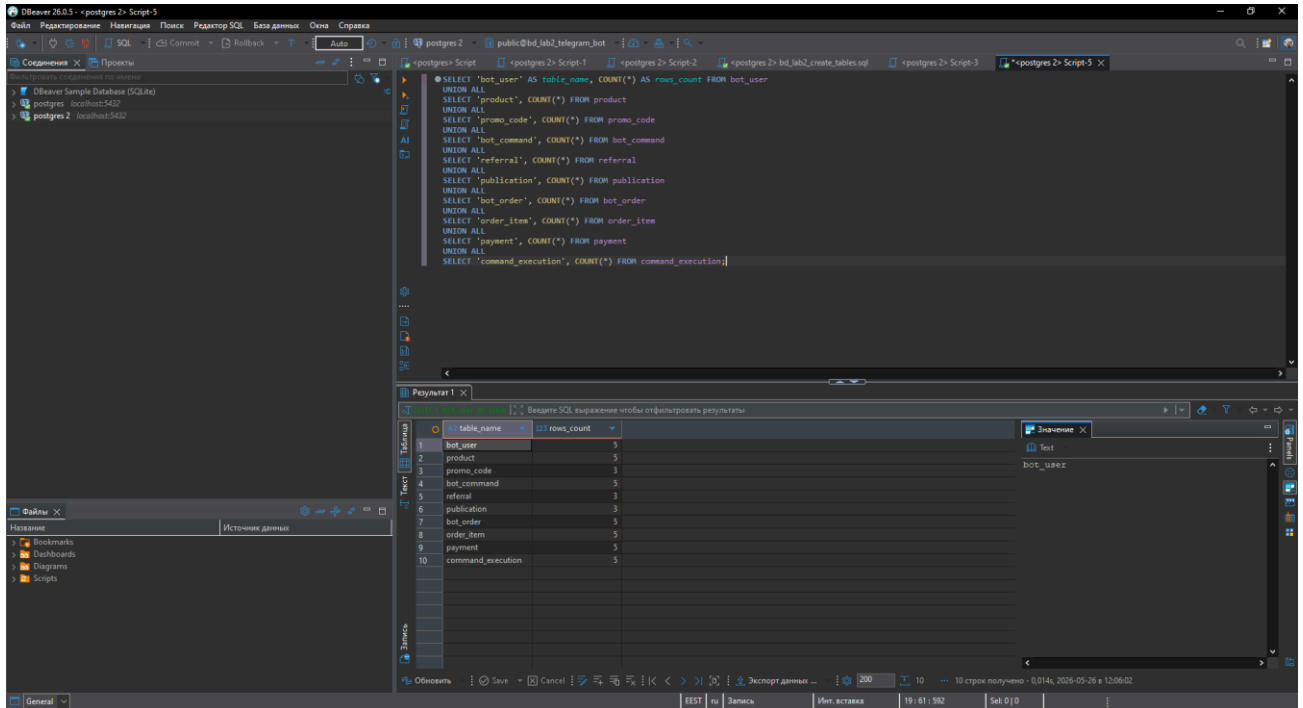
```
SELECT 'order_item', COUNT(*) FROM order_item  
UNION ALL
```

```
SELECT 'payment', COUNT(*) FROM payment
```

UNION ALL

```
SELECT 'command_execution', COUNT(*) FROM command_execution;
```

У результаті перевірки було встановлено, що всі таблиці містять тестові записи, необхідні для подальшого виконання запитів.



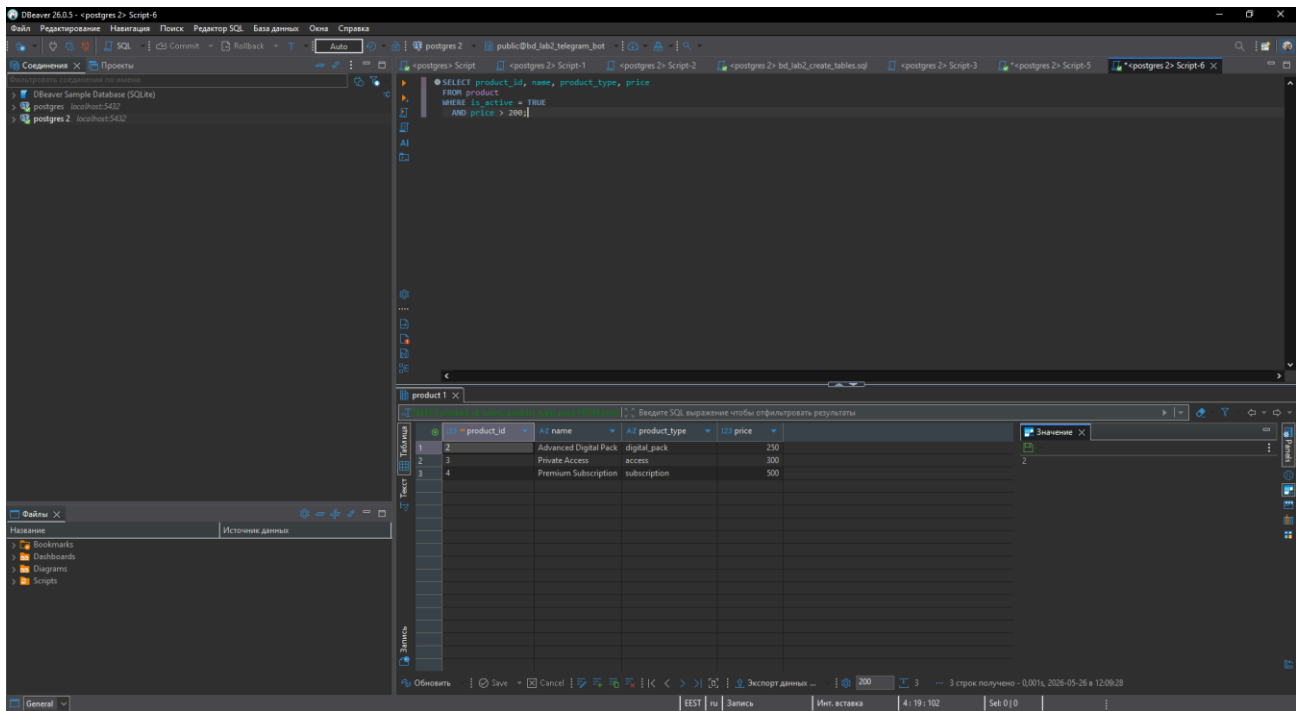
Виконання SELECT-запитів

Для отримання даних з таблиць були виконані запити SELECT з фільтрацією через WHERE та вибором конкретних стовпців.

Вибір активних товарів дорожче 200

```
SELECT product_id, name, product_type, price  
FROM product  
WHERE is_active = TRUE  
AND price > 200;
```

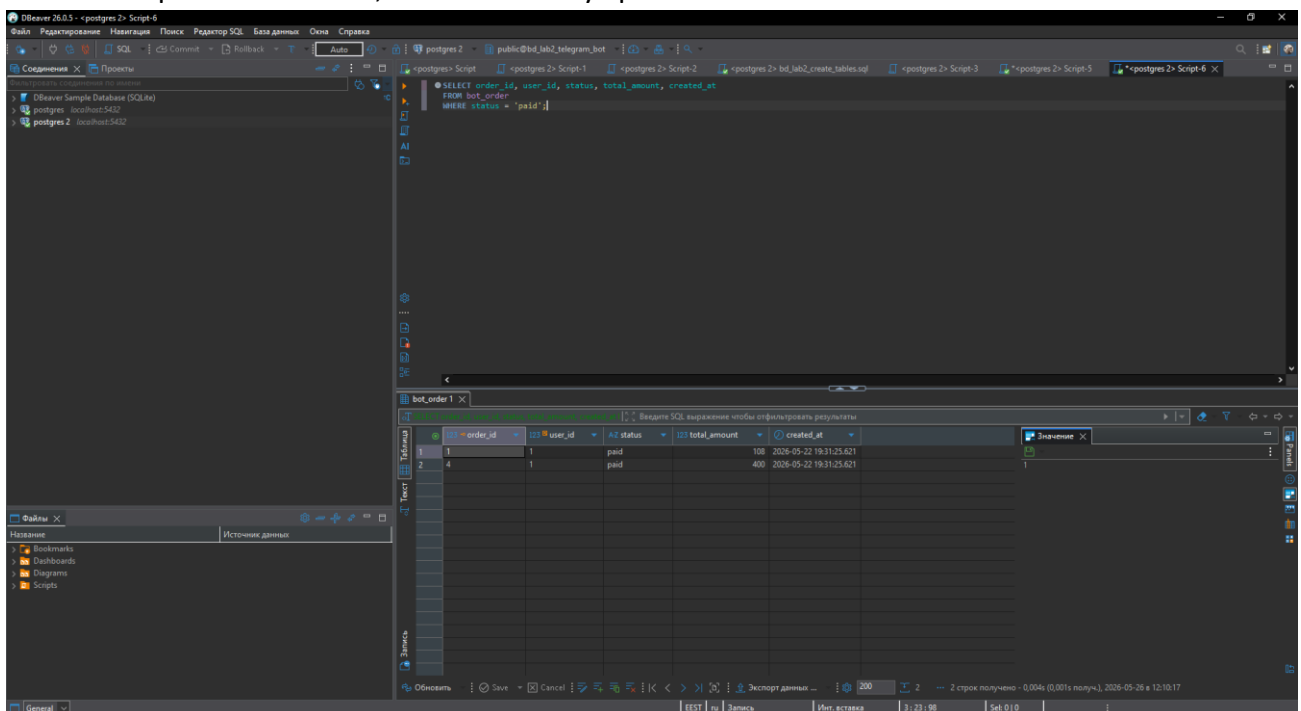
Запит вибирає активні товари, ціна яких перевищує 200. У результаті було отримано список товарів, які відповідають заданій умові.



Вибір оплачених замовлень

```
SELECT order_id, user_id, status, total_amount, created_at
FROM bot_order
WHERE status = 'paid';
```

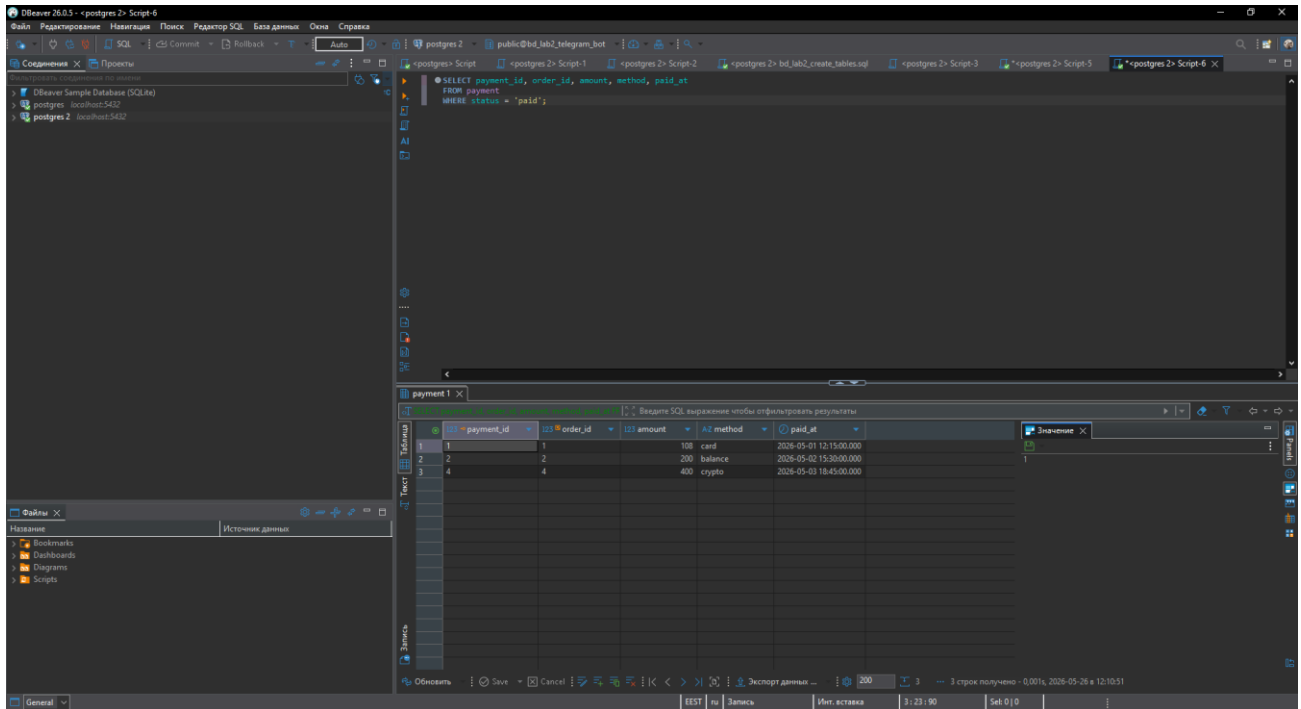
Запит вибирає замовлення, які мають статус paid.



Вибір успішних платежів

```
SELECT payment_id, order_id, amount, method, paid_at  
FROM payment  
WHERE status = 'paid';
```

Запит вибирає платежі, які були успішно оплачені.



Виконання INSERT-запитів

Для перевірки додавання нових записів було створено нового користувача, товар, замовлення, позицію замовлення та платіж.

```
INSERT INTO bot_user (user_id, telegram_id, username, role, status,  
balance)  
VALUES (10, 100000010, 'new_test_user', 'user', 'active', 50.00);
```

```
INSERT INTO product (product_id, name, product_type, price, is_active)  
VALUES (10, 'Test Digital Product', 'digital_pack', 450.00, TRUE);
```

```
INSERT INTO bot_order (order_id, user_id, promo_code_id, status,  
total_amount)  
VALUES (10, 10, 1, 'created', 450.00);
```

```
INSERT INTO order_item (order_item_id, order_id, product_id, quantity,  
unit_price, line_total)
```

```
VALUES (10, 10, 10, 1, 450.00, 450.00);
```

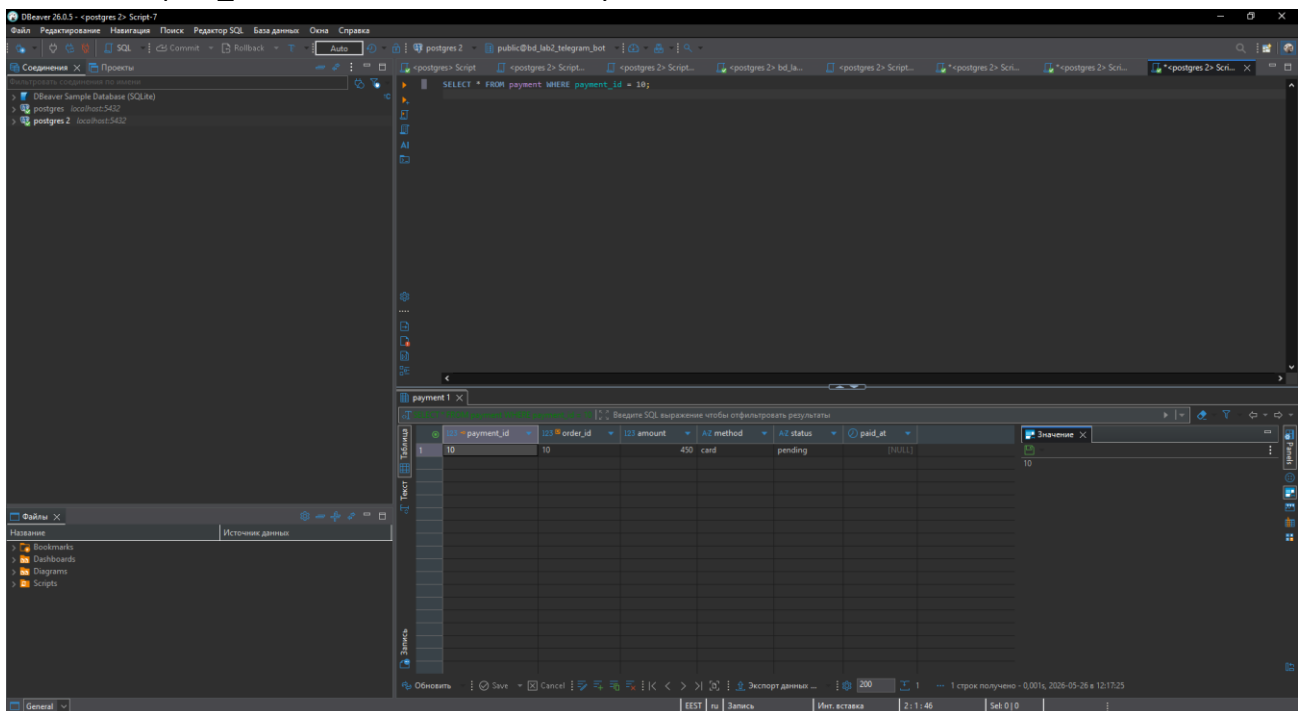
```
INSERT INTO payment (payment_id, order_id, amount, method, status,  
paid_at)
```

```
VALUES (10, 10, 450.00, 'card', 'pending', NULL);
```

Після виконання INSERT було перевірено, що новий платіж з'явився у таблиці payment.

```
SELECT *  
FROM payment  
WHERE payment_id = 10;
```

У результаті перевірки було отримано запис з payment_id = 10, статусом pending і порожнім значенням paid_at, оскільки платіж ще не був оплачений.



Виконання UPDATE-запитів

Для перевірки зміни даних було оновлено статус замовлення та платежу. Така операція імітує ситуацію, коли користувач оплатив створене замовлення.

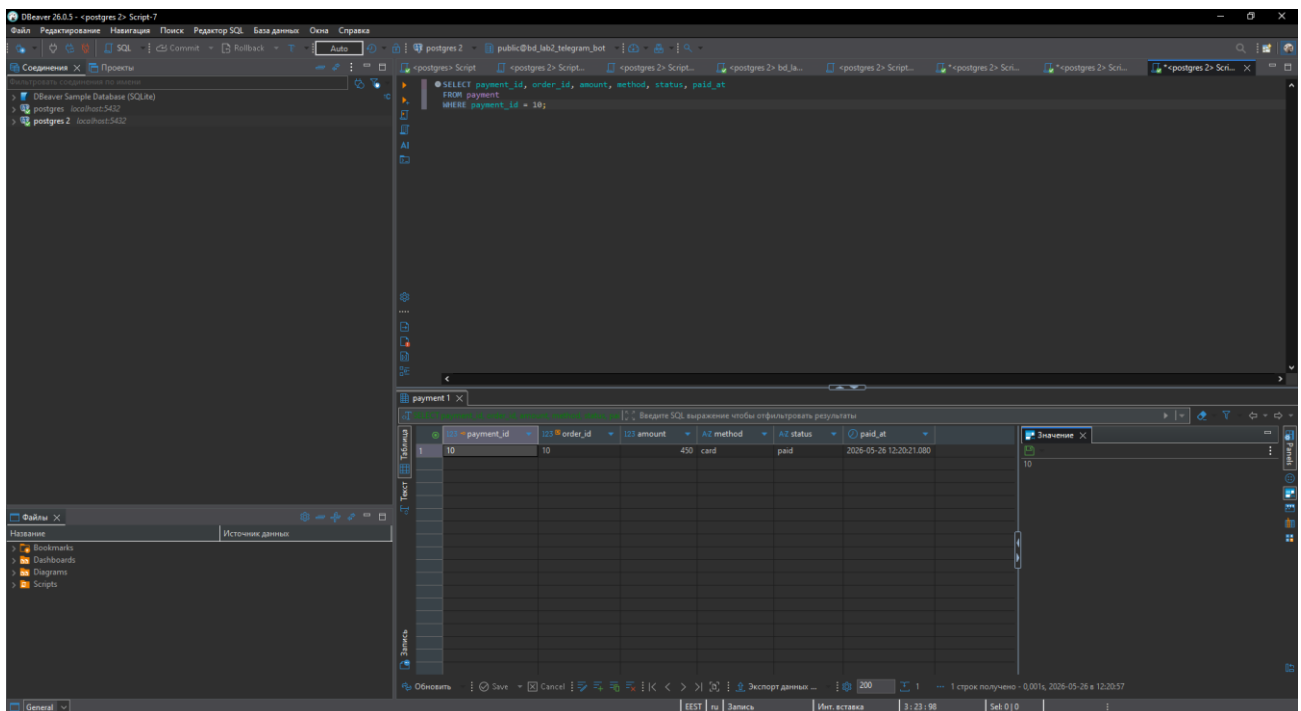
```
UPDATE bot_order  
SET status = 'paid'  
WHERE order_id = 10;
```

```
UPDATE payment  
SET status = 'paid',  
    paid_at = CURRENT_TIMESTAMP  
WHERE payment_id = 10;
```

Після оновлення було виконано перевіряючий запит:

```
SELECT payment_id, order_id, amount, method, status, paid_at  
FROM payment  
WHERE payment_id = 10;
```

У результаті статус платежу змінився з pending на paid, а поле paid_at отримало дату та час оплати.



Виконання DELETE-запиту

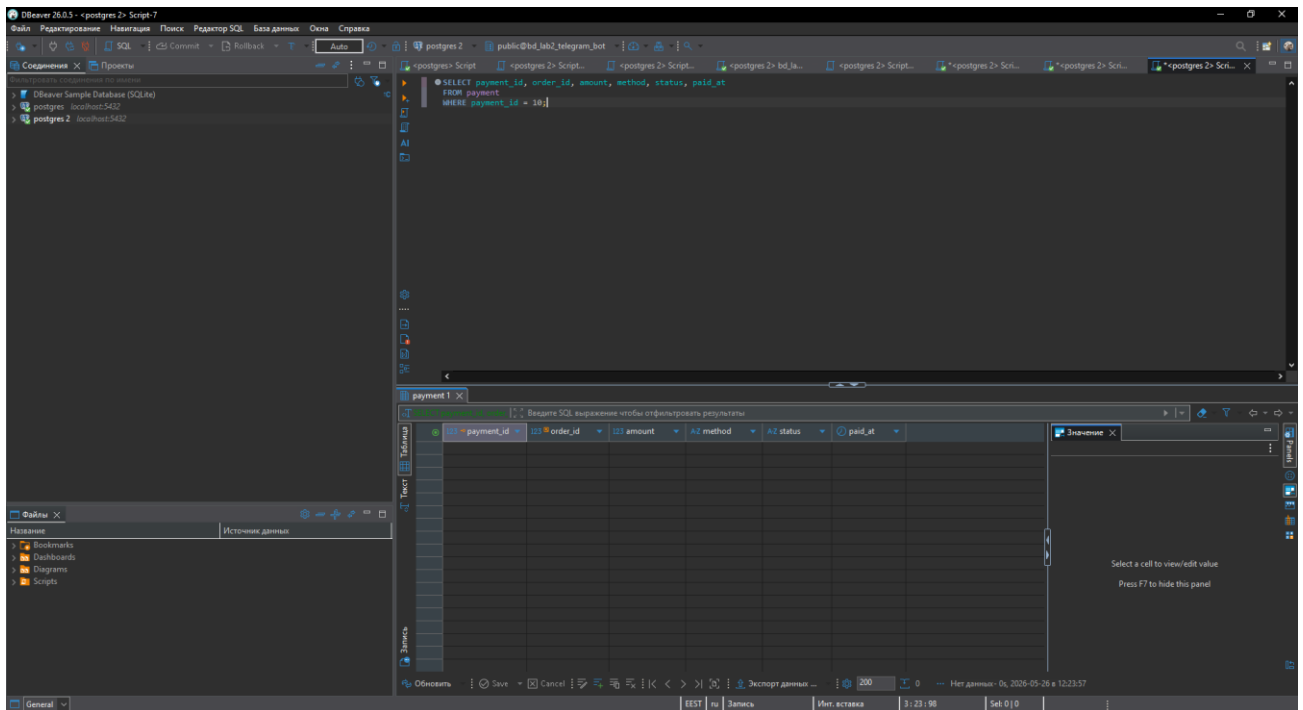
Для перевірки безпечного видалення даних було видалено тестовий платіж з таблиці payment. Видалення виконувалося з умовою WHERE, щоб не зачепити інші записи таблиці.

```
DELETE FROM payment  
WHERE payment_id = 10;
```

Після видалення було виконано повторну перевірку:

```
SELECT payment_id, order_id, amount, method, status, paid_at  
FROM payment  
WHERE payment_id = 10;
```

У результаті запит не повернув жодного запису, що підтверджує успішне видалення платежу.



Короткий опис виконаних операцій

У межах лабораторної роботи були виконані такі операції:

Тип запиту	Призначення
SELECT	Отримання даних з таблиць із фільтрацією
INSERT	Додавання нового користувача, товару, замовлення, позиції замовлення та платежу
UPDATE	Зміна статусу замовлення і платежу після оплати
DELETE	Безпечне видалення тестового платежу за конкретним ідентифікатором

У запитах UPDATE та DELETE використовувалась умова WHERE, щоб змінювати або видаляти тільки конкретні записи.

Висновки:

У результаті виконання лабораторної роботи було протестовано базу даних Telegram-бота за допомогою OLTP-запитів. Було виконано вибірку даних через SELECT, додавання нових записів через INSERT, зміну існуючих записів через UPDATE та безпечне видалення запису через DELETE. Результати виконання запитів показали, що створена в попередній лабораторній роботі схема бази даних працює коректно. Дані успішно додаються, змінюються, вибираються та видаляються з урахуванням обмежень цілісності й зв'язків між таблицями.